



ALPHA EDTECH

 DOCUMENTAÇÃO TÉCNICA — GRUPO 5

Arquitetura do Frontend: OmniConnect Mobile

Visão técnica sobre a estrutura Feature-First Clean Architecture,
padrões de comunicação e fluxo de dados do aplicativo.

 GRUPO 5 |  PROJETO OMNICONNECT

Visão Geral

O app mobile do OmniConnect é desenvolvido em **Flutter** e segue a **Feature-First Clean Architecture**. Isso significa que o projeto é organizado por funcionalidades (features), e cada funcionalidade carrega suas próprias camadas internas de dados, domínio e apresentação.

Essa abordagem foi escolhida porque:

- › Permite que múltiplos desenvolvedores trabalhem em features diferentes sem conflito.
- › Mantém o código modular e fácil de localizar.
- › Separa claramente a lógica de negócio da interface visual.
- › Facilita testes isolados por camada.

Estrutura de Pastas

```
mobile/lib/
|- app/                                ← ponto de entrada do app (MaterialApp)
|  |- app.dart
|  |- config/                          ← configurações do ambiente
|  |- routes/                          ← roteamento e navegação
|  '- theme/                          ← design system (cores, fontes)
|
|- features/
|  '- <nome_da_feature>/              ← uma pasta por funcionalidade
|     |- domain/                      ← objetos de negócio puros
|     |  |- repositories/             ← contratos (interfaces)
|     |  '- usecases/                 ← regras de negócio
|     |- data/
|     |  |- models/                  ← entidades com serialização
|     |  |- datasources/              ← chamadas HTTP à API
|     |  '- repositories/             ← implementação concreta
|     '- presentation/
|        |- providers/                ← estado e injeção (Riverpod)
|        |- screens/                  ← telas completas
|        '- widgets/                 ← componentes visuais
|
'- shared/
   |- constants/                     ← constantes globais
   |- errors/                         ← hierarquia de falhas
   |- network/                       ← cliente HTTP centralizado
   |- utils/                          ← funções utilitárias
   '- widgets/                       ← widgets reutilizáveis
```

Responsabilidades por Camada

app/ — Configuração global

Contém tudo que diz respeito ao aplicativo como um todo, não a uma feature específica.

Arquivo/Pasta	Responsabilidade
<code>app.dart</code>	Inicializa o <code>MaterialApp</code> , define tema e roteador.
<code>routes/</code>	Mapeia nomes de rotas para telas (<code>'/login' → LoginScreen</code>).
<code>theme/</code>	Paleta de cores, tipografia, estilos de componentes.
<code>config/</code>	Variáveis de ambiente, configurações por build.

shared/ — Infraestrutura compartilhada

Código que **múltiplas features utilizam**. Nenhuma feature deve replicar o que está aqui.

Pasta	Conteúdo	Exemplo
<code>network/</code>	Cliente HTTP (Dio)	<code>ApiClient</code> com JWT.
<code>errors/</code>	Falhas padronizadas	<code>ServerFailure</code> , <code>NetworkFailure</code> .
<code>widgets/</code>	Widgets genéricos	<code>LoadingOverlay</code> , <code>ErrorDialog</code> .

features/<feature>/ — Funcionalidades Isoladas

Cada funcionalidade vive em sua própria pasta (ex: `auth`, `conversations`, `contacts`).

3.3.1 1. Camada domain/ — O Coração da Feature

Regra de Ouro

Não depende de Flutter, de HTTP, nem de banco de dados. É Dart puro.

- `entities/`: Objetos de negócio imutáveis. Não possuem lógica de serialização (`fromJson`).
- `repositories/`: Interfaces (contratos abstratos) que definem as operações.
- `usecases/`: Uma classe por operação de negócio (ex: `LoginUseCase`).

3.3.2 2. Camada data/ — Comunicação Externa

Implementa os contratos do `domain` e se comunica com a API real.

- `models/`: Extensões das *entities* com `fromJson/toJson`.
- `datasources/`: Fazem as chamadas HTTP reais. Lançam exceções.
- `repositories/`: Implementação concreta. **Converte exceções em `Failure`**.

3.3.3 3. Camada presentation/ — Interface do Usuário

- `providers/`: Gerenciamento de estado e injeção (Riverpod).
- `screens/`: Telas que consomem os providers via `ref.watch`.
- `widgets/`: Componentes específicos daquela funcionalidade.

Fluxo de Dados

O fluxo segue uma direção unidirecional para garantir previsibilidade:

1. **Interação:** Usuário interage com a **Screen**.
2. **Ação:** **Screen** chama `ref.read(provider).action()`.
3. **Decisão:** **Provider** chama o **Use Case**.
4. **Contrato:** **Use Case** chama a interface do **Repository**.
5. **Execução:** Implementação do **Repository** chama o **DataSource**.
6. **Rede:** **DataSource** faz a chamada via **ApiClient**.
7. **Retorno:** A resposta sobe como `Either<Failure, T>`.
8. **Estado:** **Provider** atualiza o estado e a **Screen** re-renderiza.

Regras de Desenvolvimento

Regra	Motivação
Isolamento	domain/ nunca importa Flutter ou data/.
Tratamento	DataSources nunca retornam <code>Either</code> , apenas repositórios.
Pureza	Screens nunca contêm lógica de negócio; apenas exibem estado.
Acesso	Providers nunca acessam DataSources diretamente.

Tecnologias Utilizadas

- › **Flutter:** Framework cross-platform.
- › **Riverpod:** Gerenciamento de estado e DI.
- › **Dio:** Cliente HTTP robusto.
- › **fpdart:** Programação funcional (`Either`) para tratamento de erros.